Slovenská technická univerzita
Fakulta informatiky a informačných technológií
Ilkovičova 3, 842 16 Bratislava 4

Prenos správ protokolom UDP

Autor: Michal Kebis
Študijný program: Počítačové systémy a siete
Ročník: 2, Cvičenie: pondelok 11:00
Predmet: Počítačové siete 1
Ak. rok: 2008/09

Zadanie:

Nad protokolom UDP (User Datagram Protocol) transportnej vrstvy sieťového modelu TCP/IP navrhnite a implementujte protokol pre prenos správ medzi aplikáciami na počítačoch (posielanie a príjem správ (funkcia „chat“)). Dĺžka správy nie je limitovaná. Prijímajúca aplikácia musí byť schopná kedykoľvej správu prijať, t.j. prijímaciu stranu nie je potrebné vopred prepnúť do režimu príjmu.
Správa bude, v prípade potreby, prenášaná po blokoch, pričom maximálnu veľkosť bloku odvoďte z maximálnej dĺžky dátového poľa rámca v sieti Ethernet.
Protokol UDP je nespoľahlivý, preto vo formáte navrhnutého protokolu uvažujte aj s poľom kontrolnej sumy a poliami pre prípadné zabezpečenie komunikácie niektorou blokovou ARQ metódou. Použite kontrolnú sumu, ktorú používa aj UDP protokol (RFC 768). Zistenie chybnej kontrolnej sumy na prijímajúcej strane sa bude iba indikovať. Používateľské rozhranie musí umožňovať:

- napísať správu a vybrať cieľový počítač, na ktorom beží aplikácia, ktorej sa správa posiela,
- zobraziť prijatú správu a IP adresu počítača, z ktorej bola správa odoslaná,
- zobraziť kontrolné sumy:
    - na strane vysielača poslanú,
    - na strane prijímača prijatú a vypočítanú.

Navrhnuté a implementované riešenie musí spĺňať tieto požiadavky:

1) Prenos správ medzi aplikáciami na rôznych počítačoch s použitím kontrolnej sumy.
2) Rozdelenie správy do vhodných blokov.k
3) Indikovanie chyby v prípade jej detekcie na strane prijímača. Chyba v komunikácii bude simulovaná.

<u>V dokumentácii musí byť:</u>
- špecifikácia formátu navrhnutého protokolu s opisom jednotlivých polí a opisom činnosti protokolu
- voľba veľkosti prenášaných blokov,
- voľba prípadného zabezpečenia proti chybám a spôsobu potvrdzovania,
- uvedený spôsob simulácie výskytu chyby,
- uvedený blokový návrh riešenia,
- opísané používateľské rozhranie,
- voľba implementačného prostredia.

Formát dátovej časti UDP paketu:

Type (1B)	Length (2B)	Flags (1B)	ID (2B)	Number (2B)	Total (2B)	Checksum (2B)
Prenášané dáta obsah podľa typu (maximálne 1460B)						

Type (1B) Typ prenášaných dát

0	**Nickname**
1	**Správa**
2	**ARQ dáta**

Length (2B) Počet prenášaných znakov v hlavnej dátovej časti
Flags (1B) Flagy paketu

xxxxxxx1	fragment	značí, že bude správa bola rozdelená na viac častí ktoré treba zložiť
xxxxxx1x	acknowledge	potvrdenie o prijatí, bežne sa nepoužíva, určené pre ARQ
xxxxx1xx	ack #	Pri každom odoslanom pakete sa invertuje a slúži na odlíšenie ACK

ID (2B) Identifikačné číslo správy, inkrementuje sa po každej kompletne odoslanej správe
Number (2B) Ak sa správa nezmestí do 1 rámca, označuje číslo fragmentu aby sa dala správa zložiť v správnom poradí ako bola odoslaná
Total (2B) Celkový počet segmentov správy, využije sa pri alokovaní miesta v pamäti potrebného na zloženie celej správy a tiež na kontrolu či už prišla celá správa
Checksum (2B) Kontrolná suma z vlastnej hlavičky a prenášaných dát počítaná rovnakým spôsobom, ako pri checksume UDP hlavičky.
Dáta (max 1460B) Prenášané dáta, môže sa tu prenášať Prezývka účastníka chatu, Správa alebo jej fragment ak je dlhšia ako 1460 B, alebo ARQ dáta

Opis problému, analýza a návrh:

Zo zadania vyplýva, že budeme používať UDP (User Datagram Protocol) vnorený v protokole IP (Internet Protocol) sieťového modelu TCP/IP.

TCP/IP vrstva	Protokol
4. Aplikačná vrstva	**prenos správ**
3. Transportná vrstva	**UDP**
2. Sieťová vrstva	**IP**
1. Vrstva sieťového rozhrania	**Ethernet II**

Protokol UDP je nespoľahlivý, bez spojenia, bez potvrdenia a bez dodržania poradia paketov.

Spoľahlivosť prenosu bude zabezpečená pomocou jednoduchej kontrolnej sumy z dátovej časti. Tá bude realizovaná doplnením nulového bajtu na párny počet oktetov dátovej časti, následne sa po dvojiciach oktetov spočíta celý obsah rámca, ak je viac ako 16bitový, rozdelí sa na 2 16bitové časti ktoré sa spočítajú a urobí sa jednotkový doplnok, ktorý sa potom zapíše na koniec hlavičky. V prípade detekcie chyby pri prenose (nebude sa zhodovať kontrolná suma v pakete s kontrolnou sumou vypočítanou na strane prijímača) sa to bude indikovať vypísaním oznamu pri zobrazení správy v hlavnom okne na strane príjemcu. Odosielateľ nebude o chybe nijako informovaný, keďže nie je úlohou zadania implementovať preposielanie chybných paketov. Na druhej strane, podľa zadania sa má dať simulovať chyba pri prenose. Toto bude zabezpečené vygenerovaním nesprávnej kontrolnej sumy pri odosielaní správy (treba zaškrtnúť príslušný checkbox)

Potvrdzovanie nebude implementované, avšak riešenie bude počítať s jeho prípadným neskorším doplnením a preto bude popísané v dokumentácii a bude pripravené aj v programe.
Bude využívať tzv. Stop-and-wait ARQ metódu. Je to jedna z najjednoduchších potvrdzovacích metód. Spočíva v tom, že odosielateľ odošle vždy len jednu správu a čaká na potvrdenie. Keď dostane potvrdenie, pošle ďalšiu a zase čaká atď.. Ak nedostane potvrdenie v určitom čase, pošle danú správu znova. Na potvrdenie sa používa paket s typom 2 a flagom ACK. Na zabezpečenie aby sa nemýlili potvrdenia pri dlhej prenosovej dobe, slúži flag ACK# ktorý sa vždy invertuje a zabráni sa tým dvojitému potvrdeniu toho istého paketu ak nestihne doraziť ACK v stanovenom čase.

Maximálna dĺžka správy ktorá sa zmestí do 1 paketu sa dá odvodiť od maximálnej dĺžky dátovej časti rámca, keďže sa chceme vyhnúť zbytočnému fragmentovaniu na prepínačoch. Dĺžka dátovej časti rámca môže byť maximálne 1500 B, od tej však musíme odpočítať dĺžky všetkých vnorených hlavičiek protokolov. Tie sú za normálnych okolností 20B pre IP hlavičku a 8B pre UDP. Hlavička môjho protokolu zaberá 12B.
Celková „nosnosť“ dát teda vychádza 1500 B – 20 B – 8 B – 12 B = <u>1460 B</u>

Komunikácia bude prebiehať v móde peer-to-peer, keďže podľa zadania nemá byť použitý server. V takomto prípade sú si všetci účastníci rovní a vedia teda fungovať zároveň ako klient, ale aj ako server. Každý počítač si teda musí sám pamätať všetky údaje ako napríklad zoznam účastníkov a podobne..

Používateľské rozhranie bude grafické, implementované v MFC v programovacom prostredí VisualC++. Bude umožňovať posielanie správ, prijímanie správ, pridávanie IP adries do zoznamu kontaktov, odoberanie IP adries, výber príjemcu správy, simuláciu a detekciu prenosových chýb. Zoznam kontaktov bude realizovaný pomocou jednoduchého spájaného zoznamu.